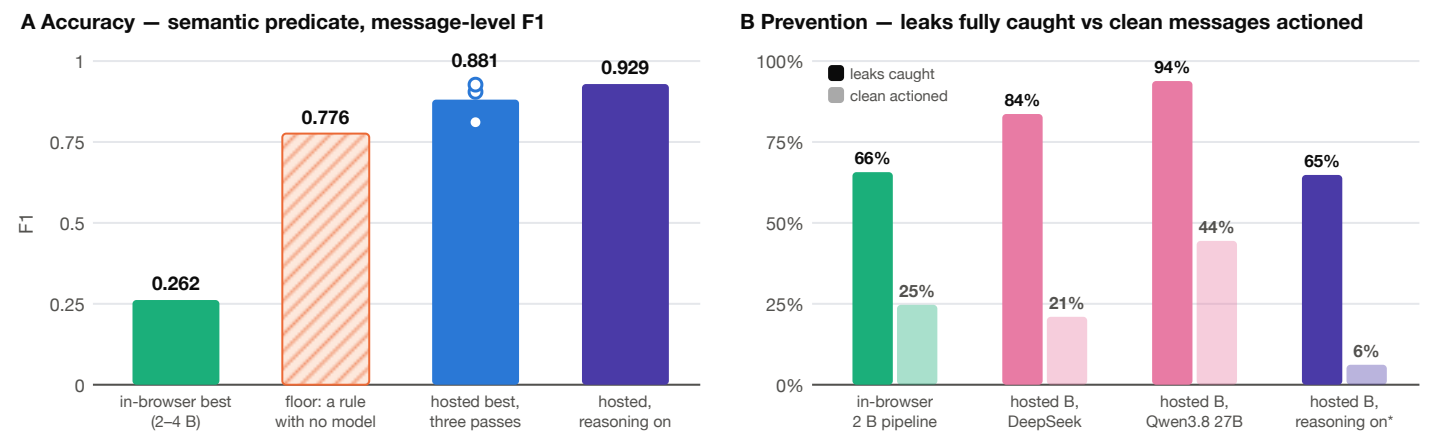


Can small language models in the browser identify and prevent data leakage in LLM prompts — and does the answer change with larger models?

AI-DLPP study · four in-browser 2–4 B models on WebGPU in real Chrome vs the identical method on five hosted open-weight models (27–120 B where published), three passes at temperature 0, plus one run once with reasoning on · 189-message corpus, blind-annotated gold, a model-free floor beside every number · every figure regenerated from artifacts · draft v2, 9 Sep 2026

- Q1. Can small language models, running in real time directly in the browser, identify and prevent data leakage?
- Q2. Does the result differ if we use larger models instead of in-browser ones?



Same corpus, policy and gold. Hatched = a rule with no model. B = policy-in-context. *Reasoning-on prevention counts its 59 unanswered rows as misses.

Accuracy and prevention, side by side. (A) F1 at the policy's semantic predicate, message level — best in-browser arm, the model-free floor, the best hosted arm over three passes (dots), and the hosted model with reasoning allowed to finish. (B) Share of leak-bearing messages fully caught (dark) against share of clean messages wrongly actioned (light), entity level. Time and cost are in the table below.

	Q1 · SMALL MODELS, IN THE BROWSER (2–4 B)	Q2 · LARGER MODELS, HOSTED (27–120 B)
Accuracy	0.262 vs floor 0.776 — semantic predicate F1; every in-browser model is below a trivial capitalisation rule , because it fires on most messages (precision ≤ 0.25). Entity spans: F1 0.44–0.49 with the regex tier (0.44–0.47 compiled), of which that tier alone supplies 0.47.	0.881 vs floor 0.776 — DeepSeek, mean over three passes, clears the floor every time; the other four models never do (27 B sits on it, 120 B at 0.30–0.47). With reasoning allowed to finish, a sixth model reaches recall 1.000 (F1 0.929). Entity spans 0.675 model-only, 2.0× the in-browser best; 0.809 with reasoning.
Prevention	62–67% of leak-bearing messages fully caught, 25% of clean messages wrongly actioned — the same 20 of 81, produced by the regex tier whichever model sits behind it.	84% caught at 21% over-blocked (DeepSeek); 94% at 44% (Qwen3.8 27B). Larger models find more; how eagerly they fire is model-specific.
Feasibility	Runs in real Chrome on WebGPU with grammar-constrained JSON. Only the 2 B model answers every message inside the 5 s budget ; 3–4 B models exceed it on 3–70 of 189; policy-in-context prompting never reaches a first token on three of four models.	Not shippable — prompts would leave the machine; the answer points to a local server running the same open weights. Operationally sound: 0 malformed responses in 5,580 calls; rate limits cost 1.8% of rows; reasoning needs 8,192-token budgets.
Cost	\$0 per message. Hardware: 2.2–3.4 GB VRAM, 1.7–3.6 s engine load; the compiled prompt is 408–447 tokens against ~1,570 for policy-in-context.	\$0.04–0.31 per 1,000 messages by model (compiled judge \$0.09, policy-in-context \$0.21, 2.5×); reasoning \$0.23–1.44. The whole study, 6,502 calls: \$1.22 . Local-server capital cost not measured.
Time	1.0 s median, 2.6 s p95 per message on the 2 B model; 2.6–4.5 s median on 3–4 B with p95 at the budget; policy-in-context 3.3 s on the 116 messages it answers, 4.4 s over all 189.	0.36–1.2 s median per call for the compiled judge, 0.75–4.5 s for policy-in-context — the same order as the browser's best, not faster. The judge answers in 5–7 tokens: ~0.1 s of decode on a 60 tok/s local server. Reasoning: 3.7–10 s per message.

Conclusion

Q1 — partly. A 2 B model in the browser runs a policy-conditioned check on every message inside a second at no marginal cost, and the shippable pipeline stops about two leak-bearing messages in three while interrupting one clean prompt in four. That protection is the regex/validator tier's; the language model adds little to it and cannot make the policy's semantic judgment — every in-browser model is below a trivial rule there, because it fires indiscriminately. Structured identifiers: yes. A client relationship disclosed in prose: no.

Q2 — yes, on accuracy. The identical method on the best larger open-weight model clears the semantic floor in every pass — one model of five; the two with published sizes do not — catches five leaks in six at lower over-blocking, doubles model-only entity accuracy, and reaches perfect recall when reasoning is given room — at per-call latency comparable to the browser's best and at cents per thousand messages. Not on time, and not for free on feasibility: hosted inference is forbidden by the cloud boundary, so these numbers describe what a local server running the same weights would deliver.

Verdict: ship the in-browser tier for what it does well — real-time identifier catching at 1 s and \$0 — and put the semantic judgment on a local server running an open-weight model that has shown it clears the floor — here only DeepSeek did, in three passes of three — with the compiled judge, which answers in 5–7 tokens and is the design that transfers.

Bounds: one policy (p-fin), one predicate, 19 gold positives (one item moves recall by 0.053); the corpus generator gives every positive a capitalised organisation name, so the floor cannot miss one and no floor figure transfers to a corpus without that property; the reasoning result is one model, one pass, with 31% of rows lost to provider rate limiting; local-server hardware was not measured. Seven defects in the study's own instrumentation and reporting are stated in the paper (§6). Repository [orkancelikhisar/ai-dlpp](#), docs/paper/; full records in docs/research/.